

Q&A

问题 1：工具池设计 —— 为什么参考 CrewAI？为什么移除 API Key / Token 工具？

 Why: 我们参考 CrewAI 并非为了照搬它的默认工具，而是为了借用其成熟的底层多智能体编排架构和标准化的工具调用协议。在构建系统时，不重复造轮子，依托 CrewAI 能够保证我们的计划（Plan）在验证时具备执行性和可扩展性。

How: 普通的 Agent 工具池（如无限制网页搜索、直接修改本地代码文件）是为 task Automation 设计的，根据我们的研究，我们希望向教学工具池改造，在机制上限制了智能体“直接把代码甩给学生”的作弊行为，强迫它们通过解释和引导来完成任务。

Do: 剔除需要个人鉴权的外部 API 工具(我们保留了 web api，保留 agent 可以 web 工具调用和搜索能力，仅仅去掉需要鉴权参数的工具调用)，1 是因为工程实现简单了，2 是为了规避开源数据集和自动化评估中的两个问题：

① **认证依赖与隐私风险**：无论是真实的教学场景，还是开源的学术 Benchmark，我们都不可能要求用户或后续的研究者去配置个人的 GitHub Token（如下图）或 Twitter API Key。这不仅存在严重的个人隐私泄露风险，还会极大拉高系统运行和数据集复现的门槛。

② **实验的可复现性**：我们希望保证的实验的公平和可复现。依赖鉴权的外部 API 返回的结果是动态变化的，且常常伴随严格的请求频次限制，比如相同的针对 RAG 的 query 在 github 上可能第一天跟第二天的搜索结果是不同的。如果在自动化跑测中因为触发限流而导致多智能体死锁，或者今天和明天查到的 API 结果不一样，我们将彻底丢掉了进行 Baseline 对比实验的基础。

```
from crewai_tools import GithubSearchTool

# Inicialize a ferramenta para buscas semânticas em um repositório GitHub específico
tool = GithubSearchTool(
    github_repo='https://github.com/example/repo',
    gh_token='your_github_personal_access_token',
    content_types=['code', 'issue'] # Opções: code, repo, pr, issue
)
```

问题 2: Ground Truth (GT) 作用 —— 如何约束 Plan? 定位是什么?



1. GT 定位/怎么来的: 在我们的系统中, Stack Overflow 的原始高赞答案 (GT) 并不是让大模型直接“抄”的答案模板, 而是作为整个教学规划的guiding target。如果说用户的画像 (Profile) 决定了教学的initial state, 那么 GT 就定义了教学的target。参考教育领域的逆向教学设计https://pdf.hanspub.org/ae2025152_101168600.pdf。系统生成的 Plan 就是连接起点和终点的“脚手架”, 防止生成的plan出现教学路径/逻辑偏离问题。

(“脚手架”: MAS系统为了帮助用户跨越其技术盲区, 而动态生成的过渡性教学步骤。系统会根据用户的水平, 决定这些步骤应该拆得有多细、需要提供多少前置知识补充。)

2. GT 在“数据集生成阶段”的作用/约束: 在利用大模型造数据 (Stage 4) 时, 如果没有 GT, 大模型在生成教学计划时极易产生幻觉 (比如强行推荐用户去学一个根本不需要的复杂第三方库)。如何: 我们将 GT 通过prompt 喂给 LLM做参考来生成plan, 要求LLM采用逆向教学设计, 参考 GT 结果, 然后再根据当前用户的 Profile (比如他是个纯新手), 把这个完美的答案“逆向拆解”成plan, 就是我们那分步式的教学步骤。GT 在这里限制无关推理扩散, 限制了 Plan 的技术边界, 保证教的东西在技术上是需求一致的而且正确的。

3. GT 在模型训练阶段的作用, 参考文档2.1, [personplan_v3](#)

4. GT 推理验证阶段: 当我们把生成的 Plan 扔给 CrewAI 引擎, 让一个模拟的学生 Agent (调用一个大模型) 去跟着步骤走一遍后, 我们如何知道这个教学计划是否真的有效? 我们的验证逻辑是: 我们将学生 Agent 走完 Plan 后的最终输出, 与原始的 SO Ground Truth 进行对比。如果学生最终写出的代码和理解与 GT 语义一致, 说明 Plan 的脚手架搭得非常成功; 如果学生中途卡死或者写错了, 说明这个 Plan 没有成功弥合 Profile 和 GT 之间的鸿沟, 需要打回重做 (之前聊过的通过reflection, 大模型作为student agent 打分然后通过阈值去判断是否reflection)。

问题 3: 依赖结构 (depends on) —— 为什么不允许环? 如何判断合理?



1. Plan 中的 depends on 字段表示什么:

在我们的数据集中, 一个 Plan 会被拆成多个 step。depends on 描述的是步骤之间的信息或结果依赖关系, 而不是执行流程的循环结构。其含义: 如果 B depends on A, 表示步骤 B 需要使用 A 的输出结果, 因此必须等待 A 完成。这里的依赖本质是数据或上下文的传递关系,

用于明确步骤之间的先后与信息使用顺序，而不是控制流（写代码-评估-返回优化，这可以通过我们refelction体现，我理解是控制流，而不应该在数据生成中）意义上的循环或迭代。

2.为什么 depends on 结构不能存在环：

如果依赖关系中出现 $A \leftrightarrow B$ 互相依赖，例如步骤 A 需要 B 的结果，而步骤 B 又需要 A 的结果，执行时就会发生死锁：A 等 B，B 等 A，两个步骤都无法开始，最终导致整个 Plan 无法推进。这不是建模偏好问题，而是执行层面的基本约束。因此，不允许环的核心原因是保证 Plan 在执行过程中始终存在可推进的顺序，而不会出现互相等待的情况。

3.depends on 合理性如何判断：

判断分为两个层面。

(1) 第一步是结构检查，只验证是否存在互相等待的环。如果 steps 之间的 depends on 可以排出清晰的先后顺序，则说明 Plan 在执行层面是可行的；若存在环，则直接判定结构不合理。

(2) 第二步是顺序合理性检查(教学逻辑检查)，即判断步骤安排是否符合学习逻辑。例如是否解释概念之前直接进入实现，或相邻步骤知识跨度是否过大。该部分判断依据来自教育领域的基本原则：Bloom's Taxonomy 用于评估认知层级递进是否合理，Cognitive Load Theory (CLT) 用于评估任务安排是否存在不必要的认知跳跃。我们根据这些原则设计prompt指导模型对步骤顺序与依赖设计进行评分，而不是依赖人工规则。

教育部分理论（**认知负荷管理 (CLT)**，**认知层级递进 (Bloom's Taxonomy)**）参考，这部分可以用来帮助构建prompt声哼plan同样可以帮我们模型验证plan以及refelction：

认知负荷Cognitive Load：

<https://www.sciencedirect.com/science/article/abs/pii/S0306457325005461?via%3Dihub>。

认知负荷论文：

<https://www.sciencedirect.com/science/chapter/bookseries/abs/pii/B9780123876911000028>

Zone of Proximal Developmen,

https://en.wikipedia.org/wiki/Zone_of_proximal_development

问题 4：Run Test 抽样 —— 如何保证代表性与可复现，以及如何判断结果的合理性

1.为什么需要 Run Test：

Run Test 的目的，是验证我们生成的 Plan 是否在真实多智能体环境中可以执行，而不仅仅是文本上看起来合理。仅依赖人工阅读或静态分析，无法判断步骤在工具调用和执行顺序上的可行性，因此需要让智能体实际按 Plan 运行一遍，观察是否能够顺利完成任务。

此外，考虑到多智能体执行成本较高，且运行全部数据对效果验证的提升并不大，因此我们选择采用抽样方式进行测试。

2. 怎么进行Run Test:

在 Run Test 中，我们采用分层抽样方式，同时考虑 query 与 learner profile 两个维度。query 用于覆盖不同复杂度的问题，profile 用于观察 Plan 是否随用户背景变化，从而保证验证具有代表性而不过度增加执行成本。

在 query 维度上，我们按照问题复杂度进行分类，例如在数据挑选阶段将 query 区分为不同复杂度等级，并从各个复杂度区间中选取样本，以避免测试结果被某一类简单或困难问题主导。这样可以保证 Run Test 覆盖不同难度的问题，而不是仅验证少数特定任务。

在 learner profile 维度上，针对同一 query 采样不同用户背景，例如技能水平或经验不同的用户，用于观察 Plan 是否随用户条件变化。对于不同 profile 下生成的 Plan，我们通过大模型判断这些因用户背景差异而产生的计划调整是否合理。在此基础上，进一步检验个性化机制是否真实存在：若不同 profile 下 Plan 表现完全一致，则说明个性化设计可能不足；若 Plan 在结构或解释方式上随用户背景发生变化，则说明系统具备适配能力。

3. 执行结果的验证:

Run Test 执行完成后，我们关注的不是单个步骤是否成功，而是 Plan 执行的结果是否符合用户的预期。具体做法是将按 Plan 执行后的最终输出，与原始 Stack Overflow Ground Truth（用户采纳的答案）进行语义层面对比。

如果执行结果与 Ground Truth 在解题逻辑或结果上保持一致，说明 Plan 在执行层面是有效的；如果智能体执行失败、中途卡住或结果明显偏离 Ground Truth，则说明 Plan 在步骤设计或依赖安排上存在问题，需要进入修正或 reflection 流程。

问题 5：如何评估 Plan 的个性化与用户匹配性？

- ◆ 在之前的工作中，我们已对 Plan 本身的个性化进行了评估，并设计了 personality 指标。具体做法是通过设计prompt让模型对生成的 Plan 进行打分，重点考察 Plan 是否围绕用户的技能、经验水平以及背景信息进行了合理调整。

(1) 优化prompt

我们在之前的prompt的基础上引入了 Bloom' s Taxonomy 与认知负荷理论（CLT）作为判断依据，使模型在评估 Plan 时不仅考虑是否存在差异，还需判断步骤组织与知识递进是否符合基本学习逻辑。例如，步骤安排是否体现由理解到应用的递进关系，以及相邻步骤之间是否知识跨度过大。该修改的目的在于使 personality 评分具备更清晰的判断标准。

此外，我们还将Ground Truth 也纳入评估prompt中，让模型评估的时候将其作为参照信息使用，用于检查 Plan 是否偏离问题本身的目标方向，以及是否覆盖 Ground Truth 中涉及的关键知识点。此处的关注点并非要求 Plan 复现 Ground Truth，而是保证个性化调整不会导致技术路径或教学重点的偏移。

(2) Case Study

为增强评估结果的可读性与说服力，我们补充 case study 作为定性说明。通过选取同一 query 并配合不同 learner profile，对生成的 Plan 进行对比分析，可以更直观地展示 Plan 如何随用户背景变化而产生结构与策略上的调整。具体做法如下：

首先会验证Plan是否能顺利运行，如果不能，则其后续个性化分析不再有意义；然后我们会将执行得到的结果与Ground Truth进行一个语义相似度的对比，判断Plan有没有偏离主题，在执行层面是否有效；最后用优化后的评估 prompt 对 Plan 本身打分，判断这些“因 profile 不同而产生的计划差异”是否合理（例如步骤拆得是否更细、解释是否更充分、是否符合学习顺序）。从这三个角度全方位的展示出我们数据集的有效性。